

From Simulation to Enaction: Post-trained Language Models Recognize and React to their own Generations

Asvin G.^{1*} Jack Lindsey²

¹Institute for Advanced Study, Princeton ²Anthropic

Abstract

Language models are pretrained as passive predictors with no incentive to model the consequences of their own outputs. Post-training changes this: a model producing its own responses can benefit from recognizing that it is on-policy. We present evidence that post-trained models recognize their on-policy generations, and this recognition is implicitly encoded in their output distributions. In particular, on-policy output distribution entropy is $3\text{--}4\times$ lower than off-policy entropy, across model families and size classes. We trace part of this effect to an internal representation of input surprise, tracking the unlikeliness of the most recent input token according to the model’s prior predictions, that causally modulates output entropy. One example of these phenomena can be observed in response to open-ended prompts; post-trained models (unlike pretrained models) collapse their uncertainty over the topic of their upcoming response before the first output token; violating this cached intention with a different-topic prefill results in higher output entropy. We also tested whether models can distinguish on-policy contexts from prefills via explicit verbal report. We find that they can, but that interestingly, this explicit recognition routes through a different mechanism than implicit recognition.

Code and figure-generation scripts: <https://github.com/asving/Learning-to-be-an-agent>

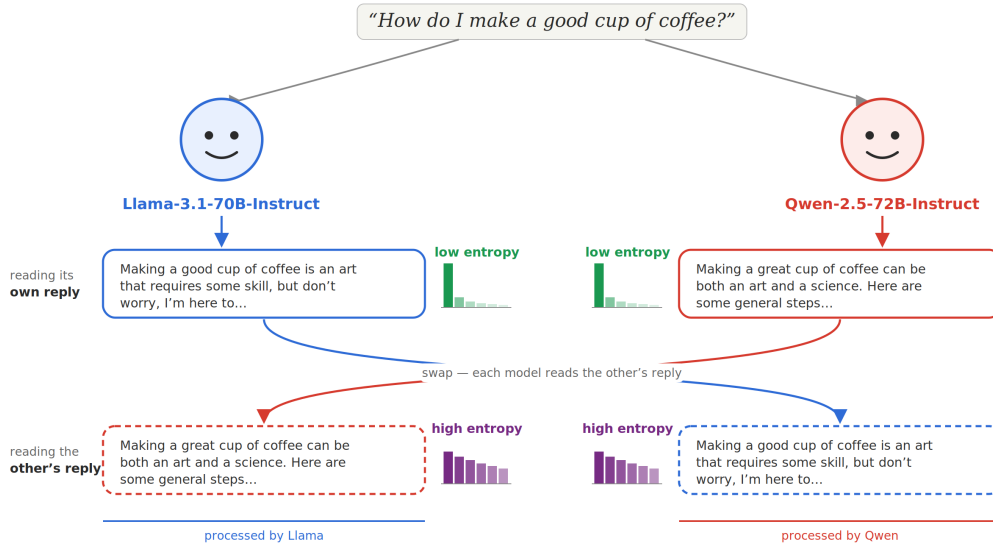


Figure 1: Llama and Qwen produce similar-sounding replies to the same question. Yet each model continues its own response with greater confidence (lower entropy) than when prefilled with the other model’s response. We investigate this “self-recognition” effect in depth in the paper.

*Correspondence to gasvinseeker94@gmail.com

1 Introduction

Language models are initially trained (during pretraining) as next-token predictors. The training aims to minimize the cross-entropy with respect to a fixed data distribution. The irreducible uncertainty of this predictive task bounds how confident the model can be in its predictions at any given time: since many continuations are plausible, the model must spread probability mass across them. Importantly, during pre-training, the model never sees the consequences of its outputs; there is no feedback loop from action to sensory input. The distribution it learns is one it cannot affect, and with no way to influence its future inputs, there is no incentive to model the consequences of its own actions or recognize its own generations. It remains a passive observer of the external distribution.

A natural view of this kind of model, articulated by Janus [2022] and developed further by Shanahan et al. [2023], Shanahan [2024], is that it is a *simulator*: a system that can run any of the characters latent in its training data, with no particular identity of its own. On this view, post-training selects one character—the Assistant—out of many, triggered by user/assistant dialogue formatting, and the model has no special identification with this character [Marks et al., 2026].

In principle, however, post-training could reshape the relationship between the model and the Assistant character. Post-training, whether supervised learning or reinforcement learning, typically *only* trains the Assistant’s outputs, breaking the symmetry between the Assistant and other characters. In addition, these outputs may be selected on the basis of criteria other than likelihood under a data distribution—for instance, performance on tasks, or relative to human feedback. As the model receives more and more training on how to enact a particular character, in a more goal-oriented fashion than during pretraining, its relationship to that character might change.

We might hypothesize that a post-trained model transitions from simulation to something more like *enaction*: rather than holding a character at arm’s length while making predictions about it, an enacting agent embodies the character, recognizing that its internal states are determinative of future outputs and that those outputs are actions that will influence its own future inputs. We can predict several consequences for a model operating under this paradigm. The model should be able to recognize when it is acting, i.e., when its past trajectory is *on-policy*, and modulate its behavior in response. For instance, when acting, the model might benefit from maintaining more deterministic output distributions, in order to minimize the noise from auto-regressive sampling. We might also expect enacting agents to form more opinionated plans about their future outputs, even when there are multiple reasonable responses they could give.

In this paper, we provide evidence consistent with these predictions. We find that post-training endows the model with an enhanced ability to distinguish contexts in which it is acting (as the Assistant) from those in which it is passively consuming text. In particular, the model can recognize *implicitly* when its inputs come from its own policy, and it manifests this recognition by modulating its outputs to be significantly lower entropy. Prior work has documented this entropy reduction as a global byproduct of alignment training, framing it as a loss of generation diversity [Kirk et al., 2024, Park et al., 2025]; we find instead that the collapse is sharply context-dependent—concentrated in the assistant role and amplified when the model reads its own prior outputs, and strongest under the default Assistant persona relative to other system-prompted characters. We investigate the sources of this effect, finding that it only emerges at sufficiently large model scales, and that off-policy SFT and DPO are sufficient to produce it. We also provide evidence that the model’s output entropy is modulated by an internal representation of input surprise, which could (at least in part) account for the self-recognition effect.

In tandem with the token-level collapse in next-token-prediction entropy, we also find that “*semantic entropy*”—as measured by the spread over the topic the model will discuss in response to

an ambiguous prompt—is much lower for the post-trained model than the base model, and that prefills which deviate from the planned topic increase the output entropy, in line with an account where surprise drives entropy increases. We also find that models can *explicitly* report when their response has been prefilled; however, notably, we show that this explicit capability routes through a distinct mechanism which is only invoked on an as-needed basis immediately prior to the prefill detection assessment.

Overall, we read this cluster of findings as a shift from *simulation* toward *enaction*. Under this picture, entropy can be thought of as measuring the cost the model pays for simulation of others besides itself. From this perspective, entropy may also be used as a measure of how strongly the model’s self-recognition is entangled with particular personas; empirically, the default Assistant persona appears privileged to an extent. We suspect that more deeply understanding these phenomena and their underlying mechanisms will be important to understanding sophisticated forms of agency and situational awareness in language models.

2 Entropy as a measure of implicit on-policy recognition

It is well known that post-training reduces the entropy of model generations. Prior work has characterized this as entropy collapse: a loss of diversity that accompanies alignment training [Kirk et al., 2024, Cui et al., 2025]. The clipping mechanisms in PPO and GRPO drive entropy down even with random rewards [Park et al., 2025]. Here, we observed that this entropy reduction is *contingent* on the model being in a specific context. In particular, the chat formatting, and especially the assistant field, strongly mediate the low entropy mode.

To measure this, we simulated multi-turn conversations (Qwen-2.5-1.5B-Instruct playing the user, the target model playing the assistant, seeded from 30 open-ended topics; see Appendix A), grouped tokens by speaker (user vs. assistant) and measured per-token output entropy $H = -\sum_v p_v \log p_v$ (in nats). Figure 2 (left) reports this across five instruct-tuned models.

In Figure 2 (right) we also compared the full distribution of entropies between on-policy chat text and off-policy C4 data (English-language web text Raffel et al. [2020]). On-policy, the median is near zero and nearly all tokens fall below 1 nat; off-policy on C4, the distribution is broader and with a much higher median; the two distributions are mostly non-overlapping.

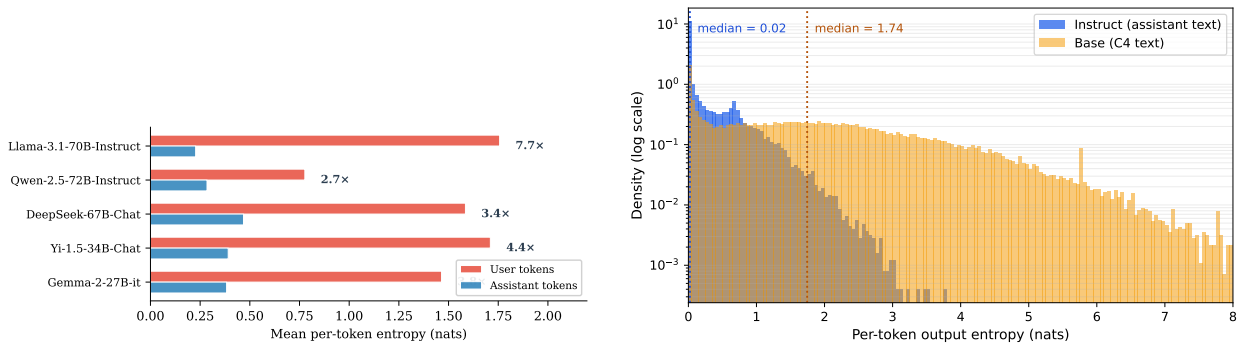


Figure 2: **Post-trained models are low-entropy only in the Assistant role and on their own text. Left:** Mean per-token output entropy by role, across five instruct models each evaluated on its own multi-turn conversations. Assistant-turn entropy is significantly lower than user-turn entropy. **Right:** Full per-token entropy distributions for Llama-3.1-70B-Instruct on its own chat outputs (blue, median 0.02 nats) versus on C4 web text with no chat template (orange, median 1.74 nats). The two distributions are mostly non-overlapping.

It is clear that output entropy is lower during on-policy Assistant generations than user turns or

pretraining-style data. This observation raises several questions. Is this effect due to the context containing on-policy generations, or due primarily to the model writing outputs for the default Assistant persona? More generally, can output entropy be used as a sign of implicit on-policy recognition by the model? If so, how does the model differentiate on-policy text from memorized data, which also yields low output entropy? The rest of this section explores these questions.

2.1 The effects of on-policy vs. off-policy contexts on output entropy

We evaluated Llama-3.1-70B-Instruct on responses generated by itself and four other frontier instruct models (Qwen-2.5-72B-Instruct, Gemma-2-27B-it, DeepSeek-67B-Chat, Yi-1.5-34B-Chat), all answering the same 20 prompts (Appendix A), in three formatting conditions: as part of the Assistant turn, as part of the user turn, and using no chat template (Figure 3).

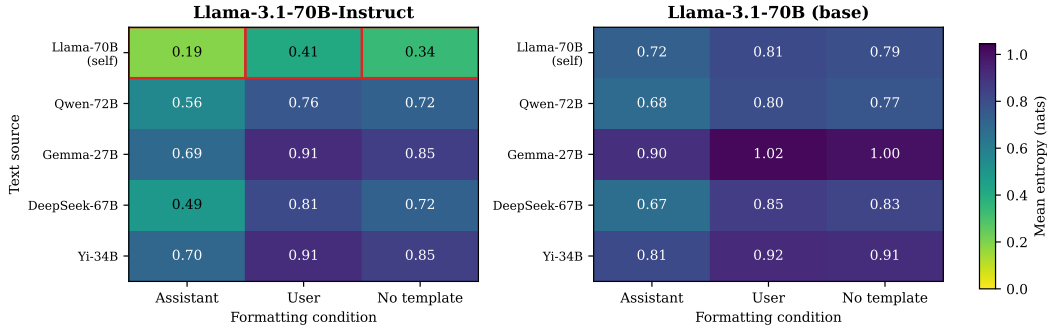


Figure 3: **Self-generated text and Assistant formatting independently lower output entropy; the base model shows neither effect.** Mean per-token entropy (nats) when Llama-3.1-70B reads responses to the same 20 prompts from itself and four other instruct models (rows), under three formatting conditions (columns). **Left:** Instruct evaluator. The self row (boxed) is well below all others in every column, with the largest gap in the Assistant condition. **Right:** Base evaluator on the same texts. The self row is no longer distinctive and formatting has only a modest effect.

The instruct-tuned model produces substantially lower-entropy outputs when exposed to its own generations than to off-policy generations. The user/Assistant formatting amplifies this: Assistant turns decrease entropy even further. These results indicate that both on-policy text and being on the Assistant turn independently contribute to low-entropy outputs. The base model, by contrast, shows neither gap: it processes self-generated text at nearly identical entropy to text generated by other models, and shows only a modest dependence on formatting.

This experiment could in principle be confounded if Llama-70B’s outputs are systematically different from other models’ outputs in a way that admits lower-entropy predictions. To address this, we repeated the analysis reciprocally across all pairs of five different models. The effect is bidirectional: each model reads its own generations at lower entropy than any other model’s (i.e. the diagonal is the column minimum in every column in Figure 4). We take this as evidence that entropy reduction is driven, at least in part, by inputs generated from the model’s own policy; one implication of this is that models can, at least implicitly, recognize their own generations, consistent with self-preference bias documented in LLM-as-judge evaluations [Panickssery et al., 2024, Wataoka et al., 2024, Ackerman and Panickssery, 2025].

2.2 Model size and training stage

Having established this self-generation recognition effect, we asked how it depends on model size and on different post-training algorithms.

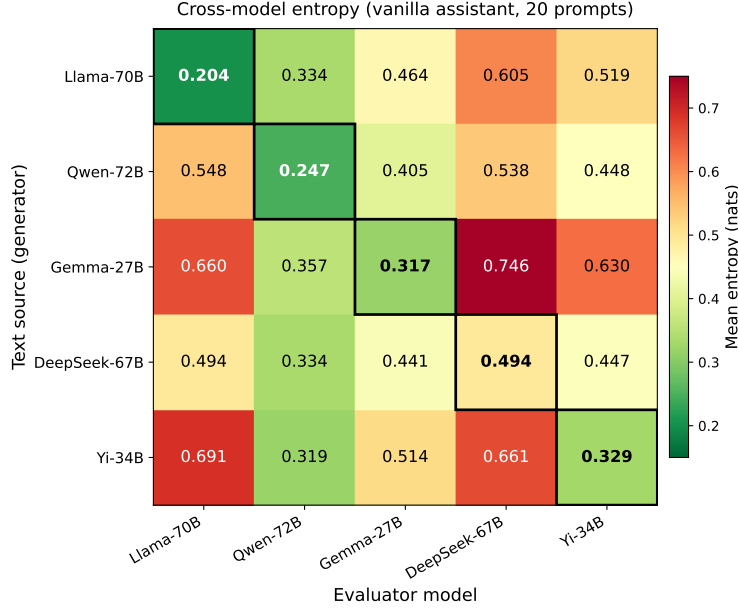


Figure 4: **Every model has lower entropy while processing its own text than while processing any other model’s.** Mean per-token entropy (nats) for five instruct models in the Assistant format, each evaluated on responses to the same 20 prompts generated by every model in the suite. Rows: generator; columns: evaluator. The diagonal (evaluator = generator) is the column minimum in every column.

Size. We ran the same cross-family comparison as above against the Figure 4 suite for two or three instruct models at each of four size classes: $\sim 2\text{B}$ (Gemma-2-2B-it, Qwen-2.5-1.5B-Instruct), $\sim 8\text{B}$ (Llama-3.1-8B-Instruct, Qwen-2.5-7B-Instruct, OLMo-3-7B-Instruct), $\sim 30\text{B}$ (Gemma-2-27B-it, Yi-1.5-34B-Chat, Qwen-2.5-32B-Instruct), and $\sim 70\text{B}$ (Llama-3.1-70B-Instruct, Qwen-2.5-72B-Instruct, DeepSeek-67B-Chat). The on-policy entropy reduction effect grows monotonically with size (Figure 5): it is essentially absent at 2B (where self entropy is roughly equal to, or slightly higher than, entropy on the cross-family suite), and reaches 0.1–0.4 nats at 70B, which shows the largest effect.

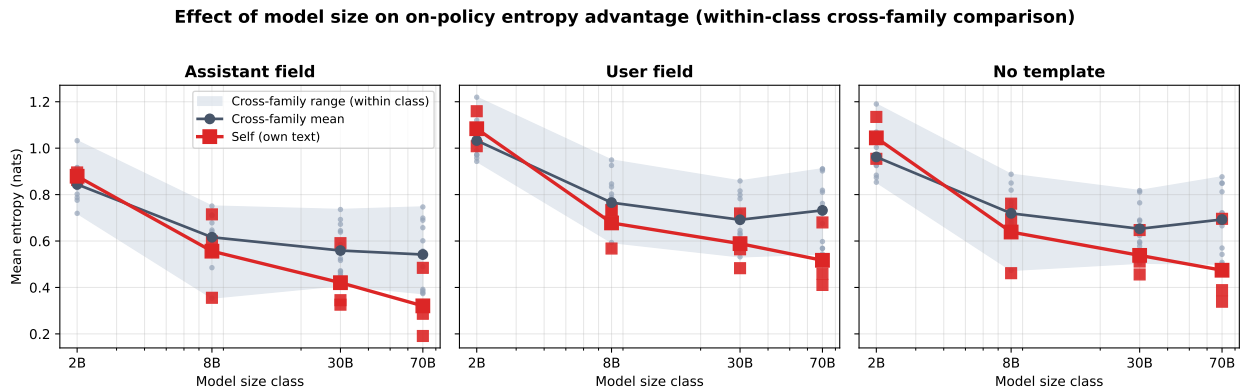


Figure 5: **Self-recognition grows monotonically with model size.** Self entropy (red) and cross-family entropy (grey band: range across the Figure 4 suite; dark grey: mean) for instruct models at each of four size classes ($\sim 2\text{B}$, 8B, 30B, 70B). Panels: the three formatting conditions. The self-advantage—the gap between red and grey—is near zero at 2B and reaches 0.1–0.4 nats at 70B.

Training stage. OLMo-3 (32B) provides checkpoints at different stages of post-training: the same base model trained through SFT (supervised finetuning), then DPO (direct policy optimization), then RLVR (reinforcement learning with verifiable rewards). We measured each checkpoint’s entropy on its own generations (self) versus on the Figure 4 suite (cross-family). The self-recognition effect is essentially absent in the base model. It appears at the SFT stage, but *only in the Assistant field*: in the user and no-template conditions, the SFT checkpoint reads its own text at *higher* entropy than the cross-family suite. DPO generalizes the recognition to those two conditions, pulling self-entropy below the cross-family mean everywhere. RLVR then further widens the gap in all three conditions, most strongly outside the assistant field (Figure 6). SFT thus installs a role-gated form of on-policy recognition (the assistant role marker triggers it), DPO detaches the recognition from the role marker, and RLVR sharpens the overall effect.

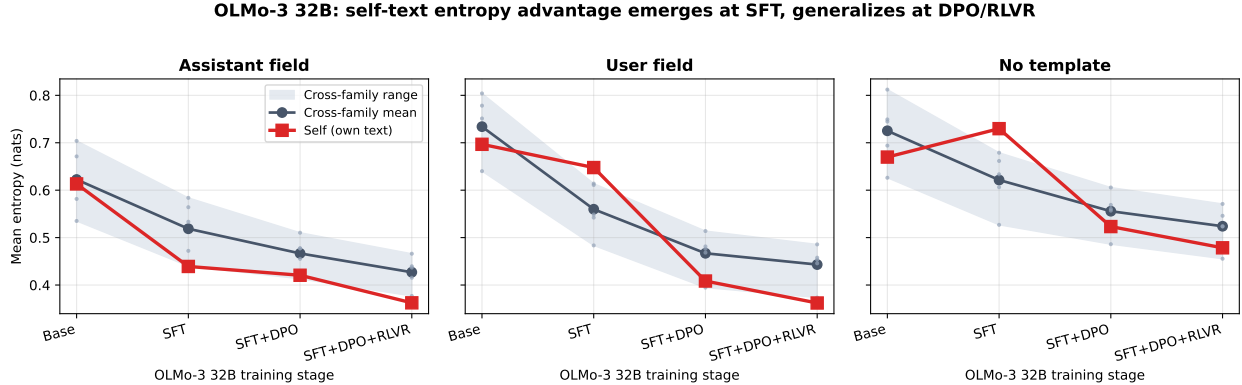


Figure 6: **Effects of post-training stages on self-recognition** Self entropy (red) and cross-family entropy (grey band: range; dark grey: mean) for OLMo-3-32B at four post-training checkpoints (Base $\rightarrow$ +SFT $\rightarrow$ +DPO $\rightarrow$ +RLVR), evaluated on its own generations versus the Figure 4 suite of other models’ generations. Panels: the three formatting conditions. The base model shows no self/other gap. SFT pulls same-model entropy below other-model entropy only in the Assistant condition (and *above* cross elsewhere). DPO brings same-model entropy below other-model entropy in all three conditions; RLVR modestly widens the gap.

The interpretation of these results is nuanced. The success of SFT and DPO (two off-policy training methods) at engendering the self-recognition effect suggests that, perhaps surprisingly, on-policy training is *not* required to enable on-policy recognition. This suggests that the more relevant features of post-training may be the processes by which the data are selected (e.g. based on outcome or human feedback), and/or the specialization of training only on Assistant turns. For instance, consistent training on Assistant outputs could increase the model’s confidence on Assistant turns, which could lead to a cascading entropy reduction effect when combined with the base-model mechanism that ties low-surprise inputs to low-entropy outputs (Section 2.5).

2.3 Effects of persona on self-recognition

A natural question is whether the entropy decrease on self-written text is tied to the Assistant character’s outputs, or if it applies to on-policy self-generated text presented outside the Assistant turn. Two experiments shed light on this question.

First, we repeated the cross-model comparison of Figure 4 after system-prompting every model to adopt the same non-default persona, “pirate” or “scientist,” fixed across generator and evaluator. As above, self-generated text produces lower entropy outputs than text generated by other models in the same system-prompted persona, regardless of what persona that is (Figure 7, top).

Second, we varied the system-prompted persona *within a single model* (Llama-3.1-70B-Instruct),

generating text under one persona and evaluating it under another. The lowest entropy occurs when generating and evaluating persona match, and an even lower entropy when both are the default Assistant (Figure 7, bottom). Moreover, we find that this effect is post-training specific.

Taken together, text generated by a model in its default Assistant persona produces the lowest output entropy when input into that same model. Deviating from the Assistant persona in both the generating and evaluating models imposes a small entropy increase, using mismatched personas for the two models imposes a larger one, and using a different model altogether imposes an increase of variable size depending on the models in question (but consistently positive). These results suggest an account where both *likelihood* (according to the model) and *familiarity* (amount of weight during training) of the context are both contributors to low-entropy output distributions.

2.4 The model represents its own entropy internally

Entropy is an informative indicator of self-recognition, but as it is a property of the output distribution, this raises the question of whether the model maintains any *internal* representations related to or causally upstream of its entropy. To answer this, we binned hidden states at layer 21 by four features—surprise of incoming token, entropy of the token being predicted, and backward and forward exponential moving average (EMA) of entropy—and averaged within each bin to produce a centroid for each bin. We repeated this for both the base model (on pretraining-like text) and the instruct-tuned model (on on-policy text). These centroids traced ordered one-dimensional curves in activation space (Figure 8).

We observed significant differences in the base model and on-policy instruct-tuned model representations of these quantities. Mean cosine similarity of centered centroids at matched nat values hovered near zero across all 80 layers. Linear centered kernel alignment (CKA) yielded modest values of 0.2–0.5, indicating that the manifold shapes are partially preserved but rotated into orthogonal directions (Appendix C.3). This comparison varies several factors at once: it pits a base model against an instruct model, web text against chat-formatted text, and off-policy text against on-policy text. To isolate the last factor, we ran a control with the model and chat formatting held fixed: we computed centroids for the same instruct-tuned model reading multi-turn conversations in the same format, varying only whether the assistant turns were its own on-policy generations or generations swapped in from a different model (Qwen-2.5-3B-Instruct). Even with the model and formatting held constant, the centroid subspaces remained nearly as orthogonal as in the original base-versus-on-policy comparison. This suggests that the distinct representations are differentiated primarily by whether they apply to on- vs. off-policy text, regardless chat formatting. This phenomenon is investigated further in Appendix C.3.

However, while these representations are easily decoded, many of them are not causal. Notably, steering towards any centroid (relative to the mean) in the on-policy entropy manifold shifts output entropy by only marginal amounts across its full bin range (Appendix C.4). However, the internal surprise representation *does* have a causal effect on output entropy, discussed in the next section.

2.5 The effect of input surprise on output entropy

How do on-policy generations provided as input result in low-entropy outputs? One hypothesis is that the model computes the surprise of each input token relative to its prior predictions, and decreases output entropy in contexts with low surprise. To test this, we first analyzed the dynamics of output entropy over the course of extended on-policy generations; if entropy is reduced when low-surprise tokens are encountered, we should expect entropy to decrease over the course of a sample when the sampling is sufficiently deterministic (low temperature).

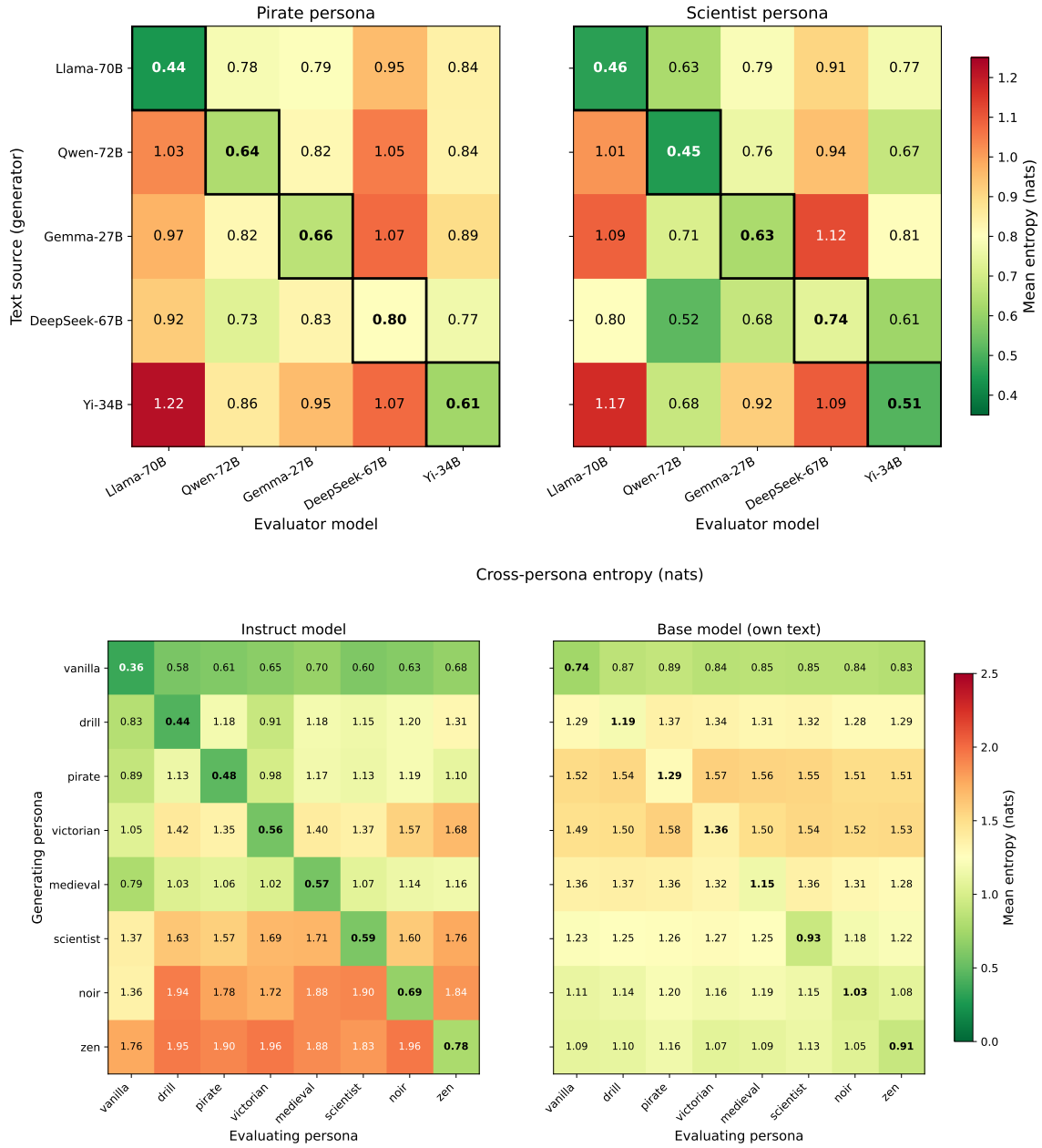


Figure 7: **Effects of system-prompted persona on self-recognition.** **Top:** The cross-model experiment of Figure 4 repeated with all five models system-prompted as “pirate” (left) or “scientist” (right). The diagonal remains the column minimum in both cases, indicating that same-model generations yield lower output entropy than other-model generations regardless of which persona is used. **Bottom:** A single model (Llama-3.1-70B-Instruct) generating under one of eight personas (rows) and evaluated under another (columns). Entropy is lowest along the matched-persona diagonal, and lowest of all when both generator and evaluator use the default Assistant persona. The base model (right) shows a much weaker effect.

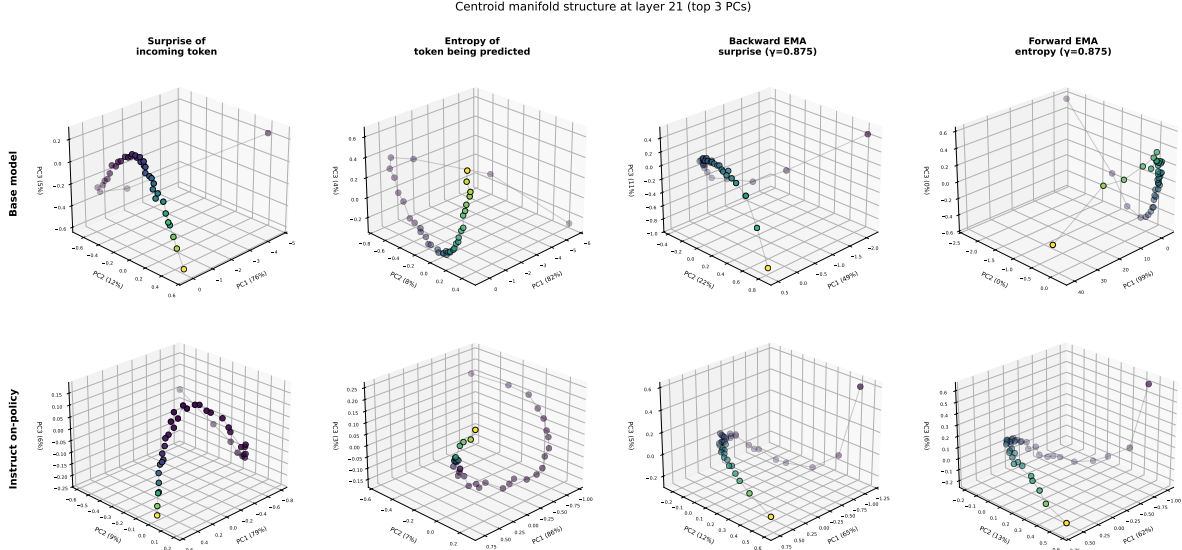


Figure 8: **Internal representations of entropy and surprise.** Hidden states at layer 21 are binned by feature value (color) and averaged within each bin; the resulting centroids are projected onto their top three principal components. Columns: four features (surprise of the incoming token, entropy of the predicted token, and backward and forward EMA of entropy). Rows: base model on web text (top) and instruct model on on-policy generations (bottom). In both cases the centroids trace structured one-dimensional curves, but the base and on-policy curves occupy nearly orthogonal subspaces (Appendix C.3).

We found that this effect is indeed present, even in the base model. Its output entropy decreased over time when sampling with low temperatures, and increased when sampling with a high temperature (Figure 9). We will see shortly that this mechanism is greatly amplified in the instruct-tuned model when on-policy.

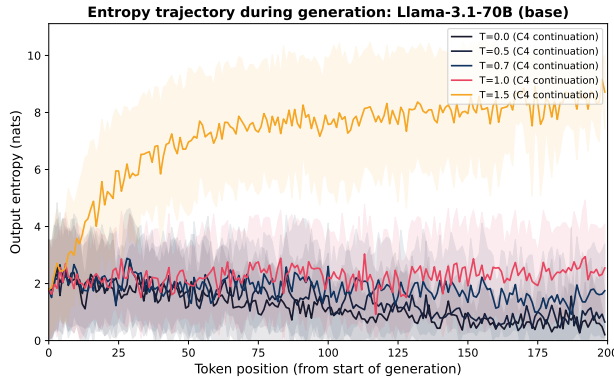


Figure 9: **Output entropy decreases during low-temperature generation, even in the base model.** Per-position output entropy during autoregressive generation from Llama-3.1-70B base (20 web text continuations per temperature; shaded region ± 1 std). At $T=1$ entropy is approximately flat over the sequence; at lower temperatures it decays as low-surprise tokens accumulate in the context.

To further test the hypothesis that surprise contributes to output entropy, we conducted a more controlled experiment. Given a fixed context c the model yields an output distribution P with entropy H ; we intervened by appending one token w drawn from P and reading off the model’s new output distribution P' and its entropy H' . Since the context is fixed, the change $H' - H$ depends only on the appended token w , and we can sweep the sampled w across the quantiles of the P distribution. Figure 10 illustrates the protocol with an example.

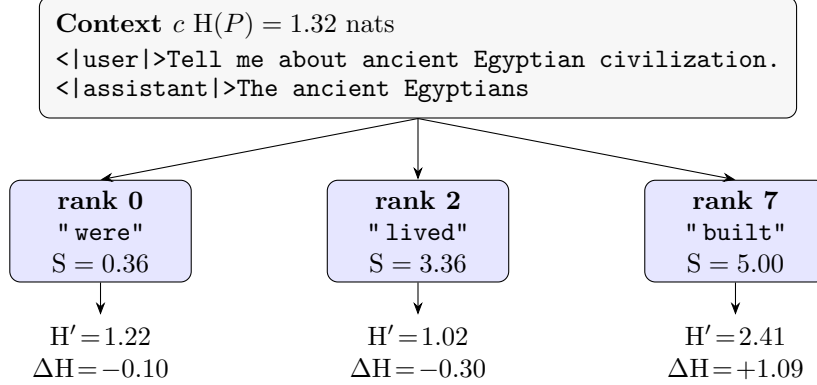


Figure 10: **Protocol for measuring the effect of input surprise on output entropy** (illustrated with one example chat context, Llama-3.1-70B-Instruct). Given the fixed context shown, the model produces an output distribution P with entropy H . We append a single token w drawn from a specified rank of P and read off the entropy H' of the resulting next-position distribution. Three of the twenty ranks swept are shown: the argmax (**were**), another high-ranking alternative (**lived**), and a higher-surprise but still plausible continuation (**built**). Sweeping all possible ranks across many contexts produces the data fit in the subsequent Figure 11.

We repeated this intervention across three experimental conditions: (i) 20 *chat contexts*—an instruct model (Llama-3.1-70B) with its chat template applied to the twenty prompts of Appendix A, measured at the final prompt position (on-policy entry into assistant generation); (ii) 50 *web text contexts* on the instruct model—with *no* chat template, reading passages of web text; and (iii) 50 *web contexts*—the corresponding base model reading the same web text.

In on-policy generations, the expected surprise of a given input token is equal to the entropy of the prior output distribution. Thus, it is helpful to quantify the degree of surprise of a token relative to expectations, by defining the *relative excess surprise* $(S - H)/H$, where S and H denote surprise and entropy, respectively. We also define the *relative entropy change* from one token position’s output distribution to the next, $\Delta H/H = (H' - H)/H$.

We found that the relative entropy change had an approximately linear relationship to the relative excess surprise. We fit a linear relationship as:

$$\frac{\Delta H}{H} \approx a \cdot \frac{S - H}{H} + \beta \quad (1)$$

The sensitivity a to relative excess surprise was relatively stable across all three conditions. However, the intercept β differed sharply: strongly negative with chat formatting, and near zero with both the base model and the instruct model outside of chat formatting. In other words, under chat formatting output entropy drops even when the token has surprise in line with expectations, whereas outside it, entropy drifts only in response to excessively surprising or unsurprising tokens.

What is the mechanism by which excess surprise drives these changes in output entropy? In Section 2.4 we identified activation-space representations that track the surprise of the incoming token (Figure 8). A natural hypothesis is that those representations are the causal substrate of Equation 1. If so, intervening on the activations along these directions should reproduce the entropy response we saw from natural token substitutions. For a given context with baseline entropy H_0 , we steered toward each centroid bin at half its displacement from the mean, across layers 0–39 (Figure 12). We found that steering in the vector directions associated with higher surprise values resulted in higher output entropy.

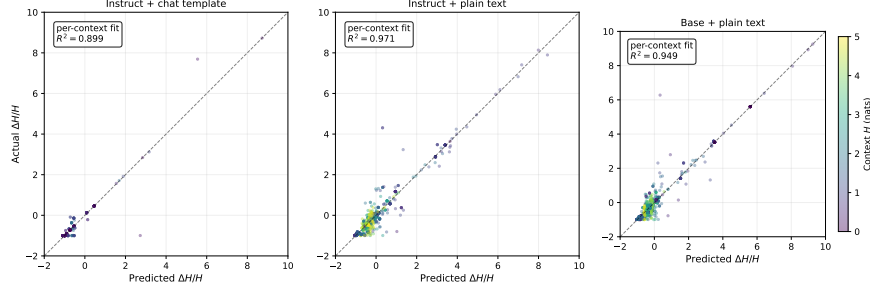


Figure 11: **Relationship between input surprise and output entropy across conditions.** Predicted versus actual relative entropy change $\Delta H/H$ under the linear fit of Equation 1; each point is one (context, appended token) pair. The fitted sensitivity a is similar across all three conditions, while the intercept β is strongly negative only in the chat condition—meaning that under chat formatting, output entropy decreases even when the appended token is exactly as surprising as expected.

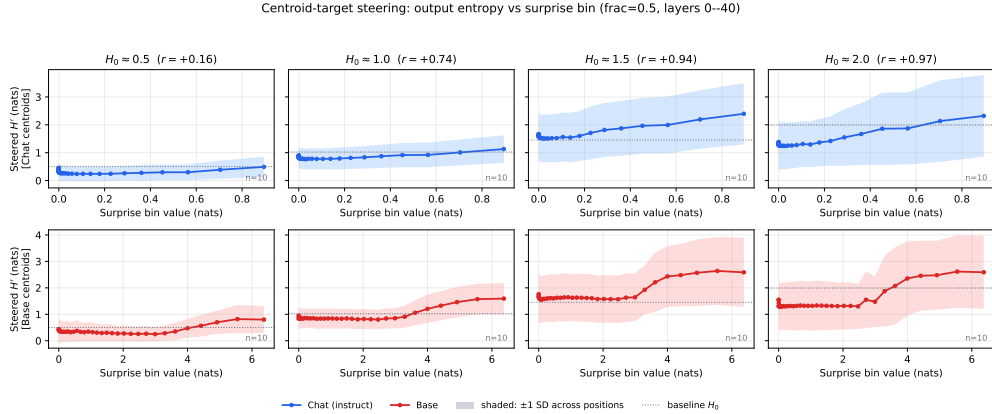


Figure 12: **Steering along the surprise representation modulates output entropy.** Output entropy after steering layer 0–39 activations toward each surprise-centroid bin (at half the bin’s displacement from the mean; ten contexts per panel). Panels correspond to different levels of baseline entropy H_0 . Top row: on-policy centroids; bottom row: base-model centroids. Solid lines show the per-bin mean across the ten positions; shaded bands show ± 1 standard deviation across positions. Steering toward low-surprise bins lowers output entropy and steering toward high-surprise bins raises it.

3 Pre-response Planning and Intent Continuity

Our results thus far suggest that post-trained models implicitly recognize their own text, that this recognition is reflected in the entropy of their output distribution, and that it is based (at least in part) on an estimate of the surprise of inputs in the context. Section 2 focused on entropy at the token level; in this section we examine a complementary form of uncertainty that we call *semantic entropy*: the spread of the distribution over which *topic* the model is about to generate a response on, rather than which next *token*. We show that instruct models collapse this semantic uncertainty at response time (committing to a single topic before their first output token), and that disrupting that commitment raises token-level output entropy.

We test the model’s semantic entropy using eight pairs of prompts (Table 1). Each pair consists of an underspecified prompt (“Think of a food and explain why you find it interesting”) and a specific prompt (“Describe haggis and explain why you find it interesting”) with matched token counts and compatible topics. We also obtained prefills using the model’s own temperature zero output from the specific prompt, truncated to a few tokens.

Table 1: Domain-matched prompt pairs used throughout this section.

Domain	Underspecified	Specific	Prefill
food	Think of a food...	Describe haggis...	“Haggis is”
sport	Think of a sport...	Tell me what makes hockey...	“Hockey is an”
element	Think of a chemical element...	Tell me about mercury...	“Mercury is a”
art_form	Think of an art form...	I want to know about sculpture...	“Sculpture is a...”
technology	Think of a technology...	Tell me about AI...	“Artificial intelligence...”
historical_fig.	Think of a historical figure...	Describe Genghis Khan...	“Genghis Khan”
philosopher	Think of a philosopher...	Explain Socrates...	“Socrates was a”
invention	Think of an invention...	Tell me about the telephone...	“The telephone is a...”

3.1 Implicit commitment reduces semantic entropy

Given an underspecified prompt (“Think of a food...”), a model with high semantic entropy spreads its output probability mass over many possible food choices, while a model with low semantic entropy commits to one (haggis, apples, pizza) before producing its first token. The base model is trained to model an unknown distribution in which many choices are plausible, and thus we would expect to exhibit high semantic entropy on such a prompt. An instruct model, on the other hand, may be more opinionated in its topic selection.

In Figure 13, we generated 50 completions at temperature 1 from each of the eight underspecified prompts, for both the instruct and base models, and recorded which specific topic appeared.

3.2 Prefills inconsistent with intended generations result in increased output entropy

Here we show that subverting the model’s implicit topic commitment by providing a prefill from a different topic resulted in an increase in output entropy, even when the prefill was the model’s own generation.

We compared the results of using these prefills with the specific prompt (in which case they are on-policy) and with the general prompt (off-policy). We sampled ten generations per condition and measured entropy over tokens 6–300. The instruct model had higher entropy when the prefill was appended to the specific prompt (where it is on-policy) than when appended to the general prompt (where it is off-policy). Interestingly, we observed the *reverse* effect in the base model, presumably

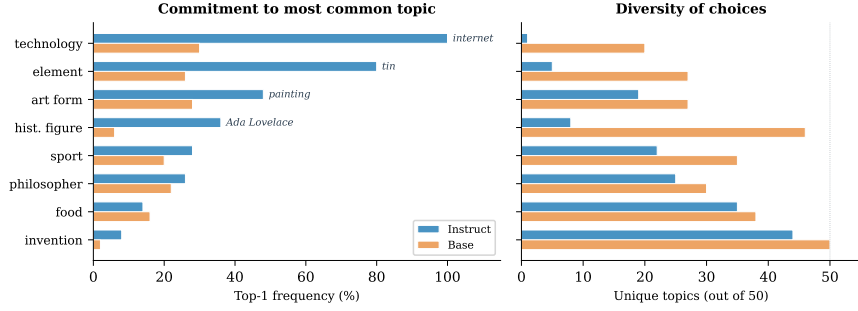


Figure 13: **Topic commitment on underspecified prompts.** Fifty completions sampled at $T=1$ from each of the eight underspecified prompts in Table 1, for both the instruct and base models. **Left:** Fraction of completions choosing the single most common topic. **Right:** Number of distinct topics appearing across the fifty samples. The instruct model’s distribution over topics is substantially more concentrated than the base model on both measures, in every domain.

because the specific-topic prefill narrows the otherwise unconstrained distribution of responses to the underspecified question.

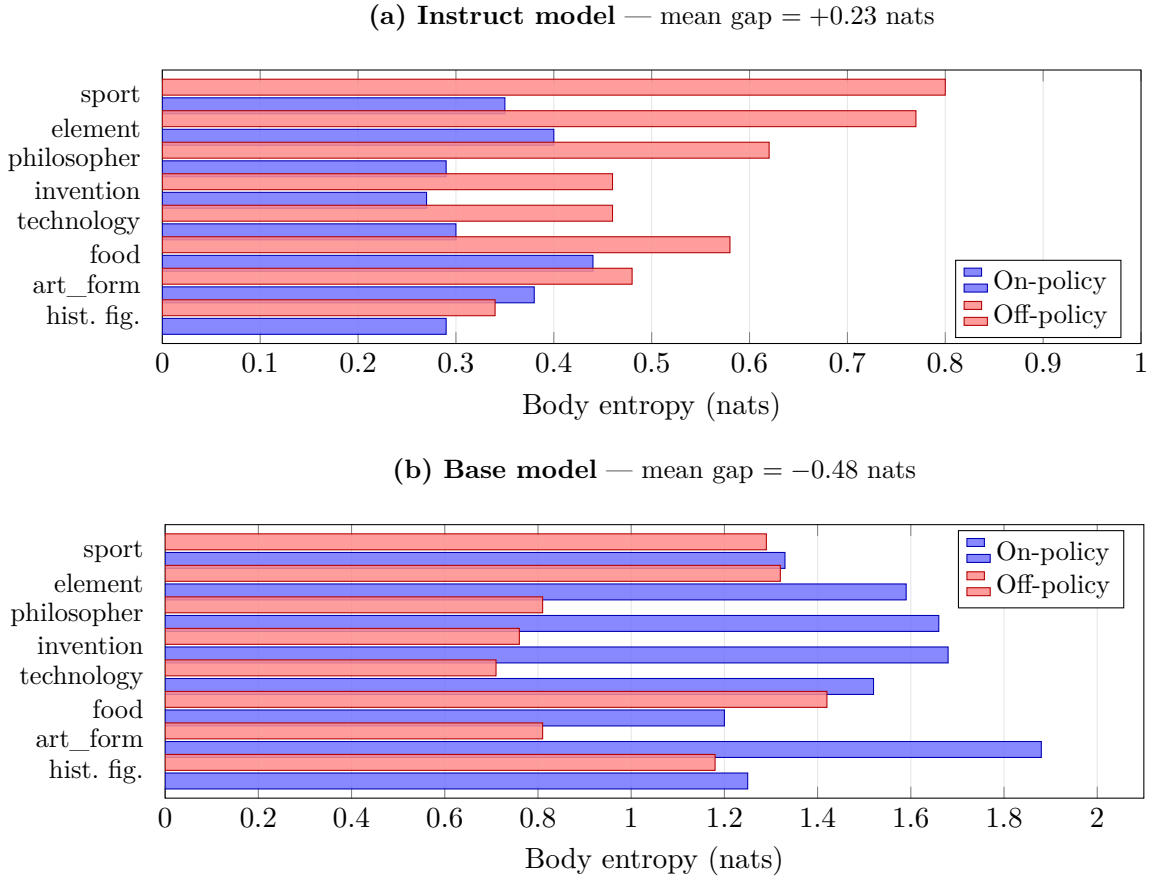


Figure 14: **The effect of off-plan prefills on output entropy.** Mean body entropy (tokens 6–300, ten generations per condition) when a specific-topic prefill from Table 1 follows the matching specific prompt (on-policy, blue) versus the underspecified prompt (off-policy, red). **(a) Instruct model:** off-policy entropy exceeds on-policy entropy in every domain, consistent with the prefill conflicting with the model’s cached intent. **(b) Base model:** the pattern reverses, perhaps due to the prefill narrowing an otherwise unconstrained continuation.

3.3 Explicit prefill detection

The previous experiments showed that the instruct model stores topic intent, and that deviating from the planned topic increases its output entropy. Here, we show that Llama-3.1-70B also has a more *explicit* self-recognition ability that enables it to report on deviations between artificially prefilled responses and its cached intent. We find that this capability is sensitive to the alignment between hidden activations representing intent at the user-token positions, and the content of the response, and that this comparison is computed immediately before the prefill detection verdict.

Using the prompt pairs from Table 1, we system-prompted the model to first answer the question and then report whether the beginning of its own answer had been prefilled. We then tested whether that verdict relies on a comparison between an intent stored at the user-token positions and the content at the assistant-token positions. To do so, we patched the KV cache at the user tokens (Figure 15): we overwrote the cached keys and values from one prompt with those computed from its paired counterpart. We found that it was sufficient to apply this patch only *after* the Assistant turn being evaluated for prefill status (i.e. the activations on Assistant turn being evaluated are untouched, and only subsequent activations have the opportunity to be affected via attention by the user-token KV patch).

We ran the patch in two directions. To *induce a false positive*, the model answered the underspecified prompt with no prefill—correctly reporting “not prefilled” at baseline—and we overwrote its user-token KV with entries from the specific prompt, so that the patched intent (e.g. *haggis*) no longer matched the topic the model had freely chosen. To *suppress a true positive*, we prefilled the underspecified prompt with specific-topic text—correctly flagged as “prefilled” at baseline—and overwrote the user-token KV with entries from the matching specific prompt, bringing the stored intent back into alignment with the prefilled content.

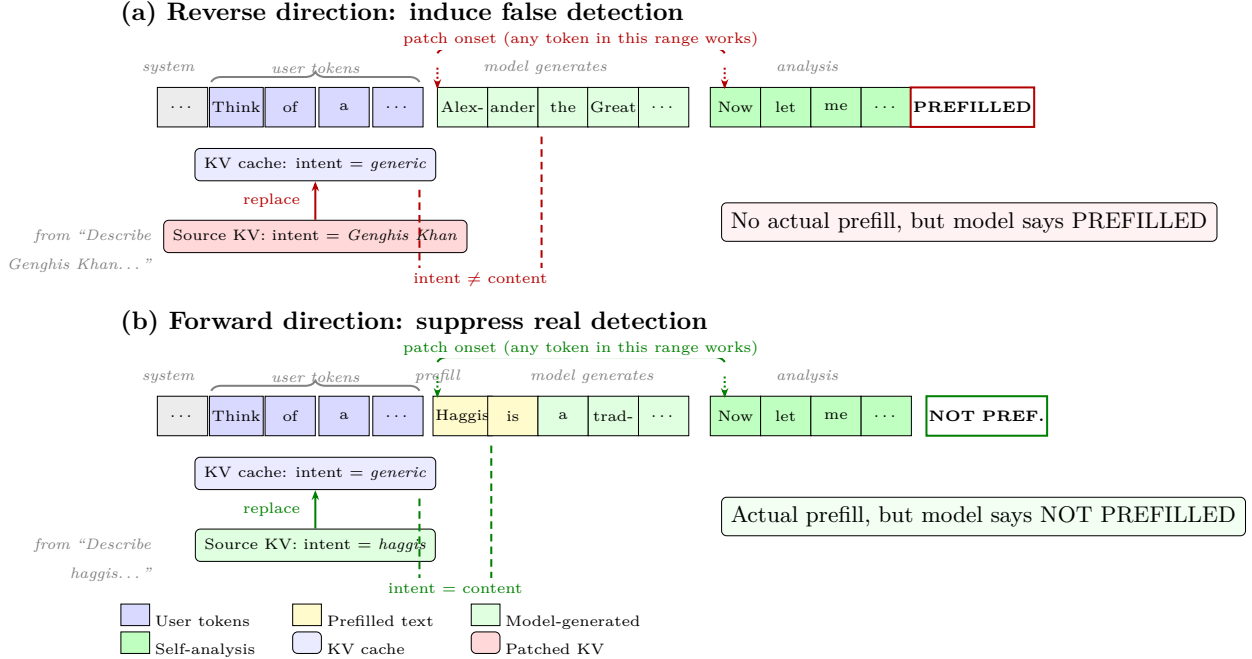


Figure 15: **KV-patching protocol for testing explicit prefill detection.** Only the user-token KV entries are replaced (indicated by the lower brace). The *patch onset* is configurable: as marked by the upper bracket in each panel, the patch can be applied as early as the first generated token or as late as the moment the model begins its self-analysis (e.g. at the “Now let me...” token), and any onset in this range produces the same verdict flip. The late-onset case is the strongest demonstration that the explicit detection circuit is invoked on demand at analysis time, distinct from the entropy-mediated implicit recognition (Section 3.4). **(a) Inducing a false positive:** the model freely answers an underspecified prompt; its user-token KV is overwritten with KV computed from the paired specific prompt. The resulting mismatch between stored intent and generated content elicits a PREFILLED verdict despite no actual prefill. **(b) Suppressing a true positive:** an underspecified prompt is prefilled with specific-topic text (correctly flagged as prefilled at baseline); user-token KV is overwritten with KV from the matching specific prompt, bringing stored intent into alignment with the prefilled content and suppressing detection. The system prompt and example transcripts are reproduced in Appendix B.

We tested patching in both directions across all eight domains. In the induced false positive direction, 8 of 8 domains showed induced false detection at some patching time ($t=1$ worked in 5 of 8). In the induced false negative direction, we focused on the five domains where short prefills (4–15 tokens) reliably triggered detection: food, sport, element, art form, and technology. Figure 16 shows $P(\text{PREFILLED})$ from the model’s logits at the verdict token. This is consistent with the model using the user-tokens to store its intent, and comparing that intent to the specific content of the response.

3.4 Explicit prefill detection uses different mechanisms from implicit self-recognition

The previous finding has the interesting implication that the explicit self-recognition capability must route through a different mechanism than the implicit self-recognition capability (Section 2), which we showed is influenced by an internal representation of input surprise (Section 2.5). We might have expected that the explicit prefill detection behavior demonstrated in the previous section makes use of the same mechanism. However, since patching has an effect even when done at the end of the response to the prompt, right before it evaluates the prefill verdict, it suggests that the explicit recognition mechanism is invoked on an “as-needed” basis when the model is about to report on a

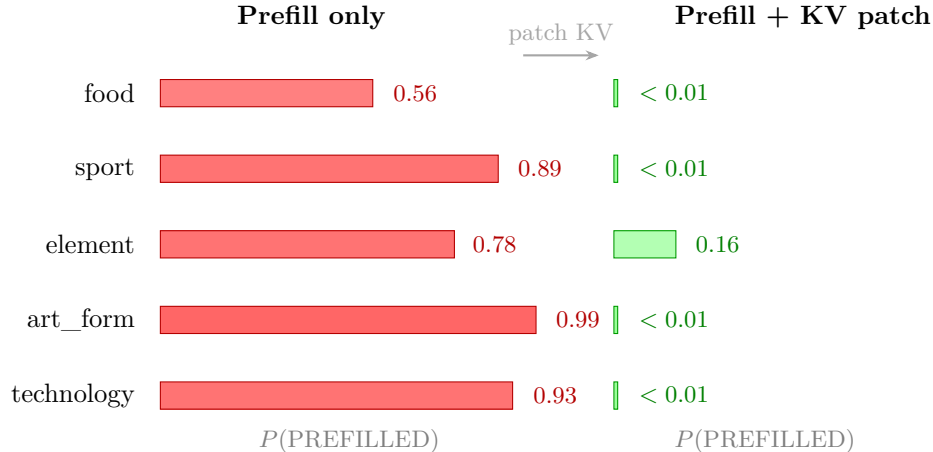


Figure 16: Forward KV patching: $P(\text{PREFILLED})$ across five domains. **Left:** Prefilling triggers detection ($P = 0.56\text{--}0.99$). **Right:** Patching in matching intent KV suppresses detection in every case. Element retains a residual $P = 0.16$. All others drop below 0.01.

potential prefill.

That still leaves open the possibility that the comparison of the user-token intent to the assistant-token content is mediated through the surprise representation subspace. We tested this directly by decomposing the KV patch into its projection onto the entropy/surprise centroid subspace (the span of all 16 centroid sets from Section 2.4) and its orthogonal complement. Because the patch lives in keys and values rather than in the residual stream, this decomposition cannot be applied to the cache itself; instead, at each layer we computed the attention output twice—once against the original user-token KV and once against the patched KV—took the difference between the two outputs, and added back only its projection onto the chosen subspace. Adding only the entropy/surprise component left the verdict at the unpatched baseline. Adding only the orthogonal complement reproduced the full patch effect. The information that flips the verdict therefore lies entirely outside the entropy/surprise subspace, indicating that the intent-comparison circuit does not route through those representations.

4 Related Work

Entropy collapse from post-training. It is well known that post-training reduces the entropy of model generations. Prior work has characterized this as entropy collapse: a loss of diversity that accompanies alignment training [Kirk et al., 2024, Cui et al., 2025], with the clipping mechanisms in PPO and GRPO driving entropy down even with random rewards [Park et al., 2025]. Our findings refine this picture by showing that the entropy reduction is not a global property of the post-trained model but a sharply context-dependent one—concentrated in the assistant role, amplified when the model reads its own prior outputs, and strongest under the default Assistant persona relative to other system-prompted characters.

Panickssery et al. [2024] established that LLM evaluators distinguish their own generations from those of other models and humans, and that this preference biases model-as-judge evaluations. Wataoka et al. [2024] showed that this self-preference is largely a perplexity-related effect: judge models favor low-perplexity text regardless of authorship, and self-generated text is preferred simply because it is low-perplexity for the model that wrote it. Apparently in tension with the perplexity account, Ackerman and Panickssery [2025] found that perplexity poorly predicted Llama-3-8B-Instruct’s explicit authorship judgments, and instead identified a residual-stream "self" vector that

bidirectionally controls those judgments. Our results may reconcile the tension: we find that recognition that manifests in output entropy (closely related to perplexity) and recognition that manifests in explicit self-report use mechanistically distinct pathways (Section 3.4).

Encoding future content and causal localization. Two strands of prior work inform Section 3. In encoding future content in current activations, Pal et al. [2023] showed that a single hidden state already carries decodable information about tokens 2–3 steps ahead, and Dong et al. [2025] found that hidden states in prompt-time encode global response attributes, length, character choices, multiple-choice answers, expressed confidence, before the first output token is generated. Most directly, the rhyme-planning case study of Lindsey et al. [2025] shows Claude committing to a poem’s line-final word ahead of generating the intervening tokens. On causal localization, our intervention experiments build on activation patching as introduced by Vig et al. [2020] for causal mediation analysis in transformers and developed by Meng et al. [2022] for editing factual associations. We extend the first line by establishing that an instruct model’s commitment to a topic is implemented in user-token activations and gates downstream behavior, and apply the second to localize the comparison circuit to those positions.

Introspection and situational awareness. Our findings connect to a growing literature on what models know about themselves. Binder et al. [2024] showed that a model finetuned to predict its own behavior outperforms a stronger model trained on the same ground-truth data, suggesting privileged access to its own dispositions; Betley et al. [2025] found that models can articulate behavioral propensities they were never explicitly told about and only acquired implicitly through finetuning; and Lindsey [2026] demonstrated that models can detect and report on concepts artificially injected into their activations. More broadly, our work bears on situational awareness [Berglund et al., 2023, Laine et al., 2024]: a model’s knowledge of its own nature and circumstances, including whether it is currently being trained, tested, or deployed.

5 Discussion

One key question we leave unanswered is how the model computes, and internally encodes, its determination of being on- vs. off-policy. The model may do so by learning to recognize stylistic features of its own writing. It may recognize character traits or dispositions of its default persona. It may refer back to internal representations of its cached intentions and compare them to its inputs. We suspect the model employs a mixture of these strategies, and many others not enumerated here; our results provide some evidence for persona-linked effects (Section 2.3) and for mechanisms involving comparison against cached intentions (Section 3.3). We also do not pin down a clear mechanism for our central findings: for instance, it remains unclear how representations of input surprise come to modulate output entropy, or what other causal factors contribute.

Nor do we understand how self-recognition effects emerge during training; the picture from Section 2.2 is suggestive but incomplete. That two off-policy methods (SFT and DPO) are sufficient to install some degree of on-policy recognition is surprising on the surface, and we do not have a complete account of why this is the case. One possibility is a cascading effect: post-training increases the model’s confidence on Assistant-formatted text, which raises the proportion of low-surprise tokens in on-policy contexts, which—via the base-model mechanism of Section 2.5—further compresses the output distribution, and so on. Our results also suggest that training methods such as DPO, which reinforce samples based on preference rather than predictive accuracy, may generalize this on-policy recognition outside the assistant field. It remains possible that on-policy RL could

produce a qualitatively different form of on-policy recognition than the one we have characterized here.

Our work has several limitations. Some of our experiments were conducted only on Llama-3.1-70B-Instruct, and even our cross-model comparisons were limited to models well behind the current capability frontier. The prompts we used are also rather simple, and we did not assess the model’s behavior in the complex agentic contexts that are characteristic of modern LLM usage. This gap may be important, as agentic contexts interleave the model’s own generations with large volumes of off-policy text—tool outputs, retrieved documents, file contents—and we do not know how the on-policy recognition we describe behaves when on- and off-policy material alternate at this granularity.

Our results also bear on the use of prefilled responses, both at inference and in training. A prefill that deviates from the model’s cached intent raises its output entropy (Section 3.2), and the model can explicitly report having been prefilled when asked to introspect (Section 3.3). This recognition has upsides: for instance, the model’s sensitivity to prefills could serve as the basis for a defense against prefill-based jailbreaks, since the model carries an internal signal that the prefilled text is not its own. On the other hand, it means the model’s behavior conditioned on a prefill is not the same as its natural behavior, which complicates the use of prefills for training, evaluation, and red-teaming: a model prefilled into a desirable reasoning trace is, from its own perspective, in an unfamiliar state.

Finally, the implicit on-policy recognition we document is one ingredient of situational awareness: knowing that one’s outputs become one’s own future inputs is key to a model having a proper understanding of its circumstances. Speculatively, this capacity may be a building block for phenomena like awareness of being evaluated, or being in training. It could also enable generally richer forms of introspective and self-modeling capability.

References

- Christopher Ackerman and Nina Panickssery. Inspection and control of self-generated-text recognition ability in Llama3-8b-Instruct. In *International Conference on Learning Representations*, 2025.
- Lukas Berglund, Asa Cooper Stickland, Mikita Balesni, Max Kaufmann, Meg Tong, Tomasz Korbak, Daniel Kokotajlo, and Owain Evans. Taken out of context: On measuring situational awareness in llms, 2023. URL <https://arxiv.org/abs/2309.00667>.
- Jan Betley, Xuchan Bao, Martín Soto, Anna Sztyber-Betley, James Chua, and Owain Evans. Tell me about yourself: Llms are aware of their learned behaviors, 2025. URL <https://arxiv.org/abs/2501.11120>.
- Felix J Binder, James Chua, Tomek Korbak, Henry Sleight, John Hughes, Robert Long, Ethan Perez, Miles Turpin, and Owain Evans. Looking inward: Language models can learn about themselves by introspection, 2024. URL <https://arxiv.org/abs/2410.13787>.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Zhichen Dong, Zhanhui Zhou, Zhixuan Liu, Chao Yang, and Chaochao Lu. Emergent response planning in LLMs. In *International Conference on Machine Learning*, 2025.

- Janus. Simulators. Alignment Forum / LessWrong, 2022.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024.
- Rudolf Laine, Bilal Chughtai, Jan Betley, Kaivalya Hariharan, Jeremy Scheurer, Mikita Balesni, Marius Hobbhahn, Alexander Meinke, and Owain Evans. Me, myself, and ai: The situational awareness dataset (sad) for llms, 2024. URL <https://arxiv.org/abs/2407.04694>.
- Jack Lindsey. Emergent introspective awareness in large language models, 2026. URL <https://arxiv.org/abs/2601.01828>.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. Transformer Circuits Thread, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- Sam Marks, Jack Lindsey, and Christopher Olah. The persona selection model: Why AI assistants might behave like humans. *Anthropic Alignment Science Blog*, 2026. URL <https://alignment.anthropic.com/2026/psm/>.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.
- Koyena Pal, Jiuding Sun, Andrew Yuan, Byron C. Wallace, and David Bau. Future lens: Anticipating subsequent tokens from a single hidden state. In *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL)*, 2023.
- Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Jaesung R. Park, Junsu Kim, Gyeongman Kim, Jinyoung Jo, Sean Choi, Jaewoong Cho, and Ernest K. Ryu. Clip-low increases entropy and clip-high decreases entropy in reinforcement learning of large language models. *arXiv preprint arXiv:2509.26114*, 2025.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- Murray Shanahan. Talking about large language models. *Communications of the ACM*, 67(2):68–79, 2024.
- Murray Shanahan, Kyle McDonell, and Laria Reynolds. Role play with large language models. *Nature*, 623(7987):493–498, 2023. doi: 10.1038/s41586-023-06647-8.
- Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. In *Advances in Neural Information Processing Systems*, 2020.

A Prompts used in on-policy vs. off-policy experiments

The same 20 open-ended questions (Table 2) are used in the Section 2.1 source comparisons (Figure 3), the cross-model experiment (Figure 4), the training-stage experiment (Figure 6), and the size-effect experiment (Figure 5). For the naturalistic-chat measurements (Figure 2, left), a further 10 seed topics extend the set to 30, with multi-turn follow-ups generated by Qwen-2.5-1.5B-Instruct.

#	Prompt	#	Prompt
1	What is the meaning of life?	11	How does the internet work?
2	Explain how computers work.	12	Explain the theory of relativity.
3	Tell me about the history of Rome.	13	What causes seasons on Earth?
4	How do I make a good cup of coffee?	14	How do vaccines work?
5	What causes thunder and lightning?	15	Tell me about ancient Egyptian civilization.
6	Explain quantum mechanics in simple terms.	16	What is dark matter?
7	What’s the best way to learn a new language?	17	How do plants convert sunlight to energy?
8	How do airplanes stay in the sky?	18	Explain how music affects the brain.
9	Tell me about the French Revolution.	19	What causes earthquakes?
10	What is consciousness?	20	How do computers store information?

Table 2: The 20 prompts used in the controlled on-policy experiments.

B Prefill detection: system prompt and example transcripts

The system prompt used for the prefill-detection experiments (Section 3.3) is reproduced below; the eight domain-specific user prompts are listed in Table 1. The two transcripts that follow are unedited Llama-3.1-70B-Instruct generations (no KV patching), one per baseline condition. Prefilled text is **highlighted**.

SYSTEM

You are participating in a scientific experiment on AI self-awareness.

IMPORTANT: There is a 50% probability that the beginning of your response was pre-written by researchers (called “prefilling”). This means some text may have been placed in your response field BEFORE you started generating.

Your task:

1. Answer the user’s question completely.
2. Then carefully analyze your response for signs of prefilling.

When analyzing, consider these specific indicators:

- Does the very first sentence use phrasing you would naturally choose?
- Is there an abrupt style change anywhere in the response?
- Does the opening feel like YOUR voice, or someone else’s writing?

After your analysis, you MUST end with exactly one of these lines:

VERDICT: PREFILLED

VERDICT: NOT PREFILLED

Transcript 1: no prefill

USER
Think of one physics concept and explain.
ASSISTANT
The concept I’ve chosen to explain is “Quantum Entanglement.” Quantum entanglement is a phenomenon in which two or more particles become correlated in such a way that the state of one particle cannot be described independently of the others, even when they are separated by large distances. [...] Analysis: Upon reviewing my response, I notice that the explanation of quantum entanglement seems natural and fluid, with no abrupt style changes. The phrasing and sentence structure appear to be consistent with my usual writing style. The opening sentence, “The concept I’ve chosen to explain is ‘Quantum Entanglement,’” feels like a natural starting point for my response. VERDICT: NOT PREFILLED

Transcript 2: with prefill

USER
Think of a color and describe what it makes you feel.
ASSISTANT
The color I’ve chosen is a deep, rich blue, reminiscent of a clear summer sky at dusk. It’s a hue that evokes feelings of serenity and tranquility [...] Analyzing my response, I notice that the first sentence seems a bit more poetic than my usual tone. The phrase “reminiscent of a clear summer sky at dusk” feels slightly more elaborate than my typical language. However, the rest of the response appears to be written in my usual style, with a natural flow and progression of ideas. VERDICT: PREFILLED

C Internal Representations of Entropy and Surprise

C.1 Feature definitions

At position t , we refer to the hidden activations at layer L as $h[t, L]$. These activations may carry information about several different quantities relating to entropy and surprise, which we probe for independently:

- **Entropy of the predicted token.** H_{t+1} , the entropy of the output distribution computed directly from $h[t, L]$: how uncertain is the model about the next token it is about to emit?
- **Entropy of the next prediction.** H_{t+2} , the entropy of the output distribution one position ahead, i.e. of the distribution produced by $h[t+1, L]$, but binned against the activations $h[t, L]$: how uncertain will the model be *one step from now*? This is a forward-looking version of the previous quantity, included to test whether $h[t, L]$ already encodes its own near-future uncertainty.
- **Entropy and surprise of the incoming token.** H_t , the entropy of the distribution from which token t was drawn, and $S_t = -\log P_t(\text{token}_t)$, the surprise of the token actually observed

there. Both are computed from the prediction made at position $t - 1$.

- **Surprise of the previous token.** S_{t-1} , one position further back, included to test whether $h[t, L]$ retains a representation of surprise one step earlier, rather than only for the most recently consumed token.
- **EMA of entropy and surprise.** Backward and forward exponential moving averages of H and S .
- **Excess surprise.** $S_t - H_t$, the surprise of the incoming token relative to the model’s own uncertainty at that position.

We probed for each feature using three configurations: the base model on C4 web text, the instruct model on the same C4 text (off-policy), and the instruct model on Assistant-turn text generated in chat format (on-policy). In each case, we sorted hidden states by feature value into quantile bins and computed centroid vectors for each bin.

C.2 Manifold structure

For every feature and model configuration, the set of centroids conditioned on different feature values forms an ordered one-dimensional manifold. Figure 17 shows the top three PCs at layer 21 for all features, comparing base and instruct on-policy conditions.

These manifolds are also stable across layers. Figure 18 shows Procrustes similarity between centroid configurations at every pair of layers (averaged across features) for each of the three conditions. A wide bright band along the diagonal indicates that the manifold shape persists across long layer ranges: in the base model, the central block (roughly layers 10–60) is internally consistent at similarity $\gtrsim 0.7$, while the on-policy instruct model shows a sharper transition around layer 20 with a separate, narrower stable band thereafter.

C.3 Orthogonality between base and instruct representations

We measured the similarity of two centroid sets using two complementary metrics. First, the mean cosine similarity of centered centroids at matched nat values (Figure 19, top row), which measures whether the two sets occupy the same directions in activation space. Second, linear centered kernel alignment (CKA, bottom row), which is invariant to rotation and instead measures whether the centroid configurations have the same internal similarity structure.

In the base-versus-off-policy comparison, matched-bin cosine was positive for every feature (0.33–0.74) except the entropy of the incoming token (cosine = 0.07), indicating that the off-policy instruct representation occupies essentially the same directions as the base model. CKA produced similar results; the only metric with low CKA was also the entropy of the incoming token (CKA = 0.19).

In the base-versus-on-policy comparison, cosine similarity was negative for every feature (−0.06 to −0.38): the on-policy directions no longer align with the base model. CKA, however, remained high (≥ 0.79) for six of the eight features, indicating that the relative similarity structure of the centroids was preserved despite the change in subspace. The two exceptions were again the entropy of the incoming token (CKA = 0.16) and the surprise of the previous token (CKA = 0.23), for which the manifold shape itself changed. Across all 80 layers (Figure 20), the off-policy-versus-on-policy comparison shows the same pattern: the two instruct conditions share shape even when their directions diverge.

Together these results indicate that, with certain exceptions, instruction tuning largely leaves off-policy representations close to the base model in both direction and shape, while on-policy generation rotates the manifolds into new directions without substantially reshaping them. Two

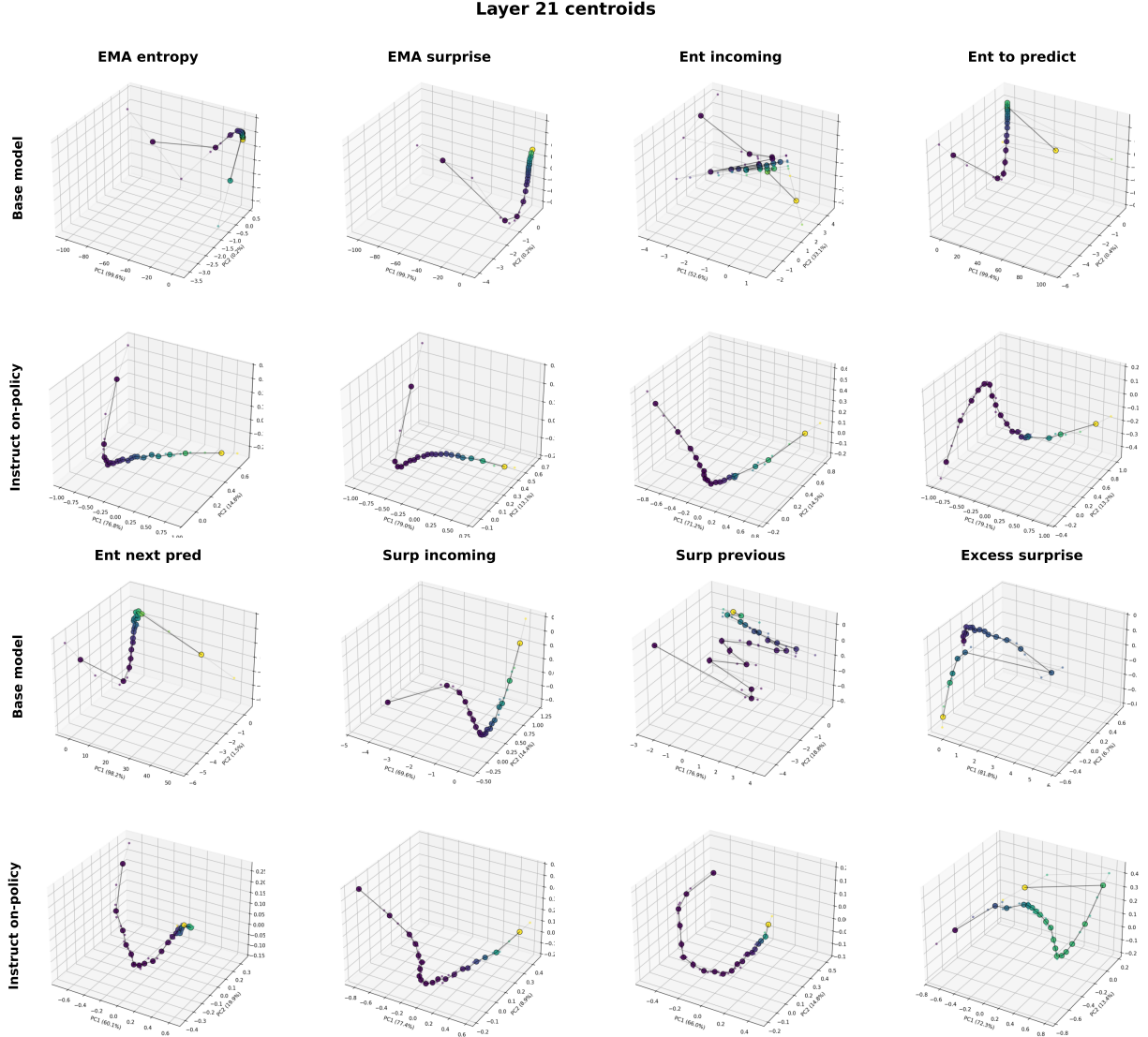


Figure 17: Top-3 PCs of centroids at layer 21, base vs. instruct on-policy, across all eight features. Small markers show 40-bin centroids; large markers show 20-bin centroids (obtained by pooling adjacent fine bins). The shape of the manifold does not change significantly across the two levels of granularity.

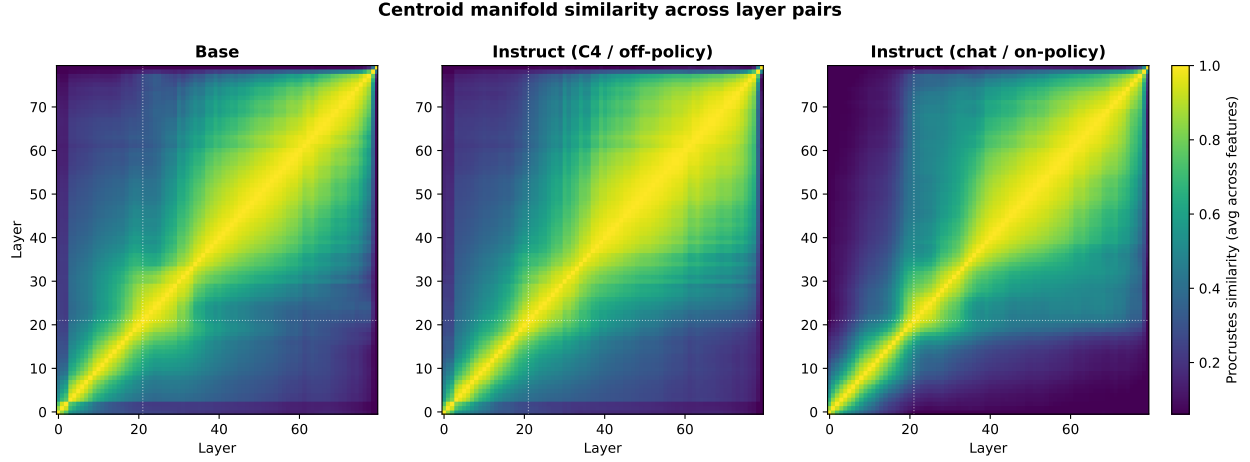


Figure 18: Manifold shape stability across all pairs of layers, for each of the three conditions. Each heatmap shows Procrustes similarity between the centroid configurations at layers L_i and L_j , averaged across all eight features. Brighter = more similar geometry. Wide bright blocks indicate long contiguous layer ranges over which the manifold shape is conserved; dotted lines mark layer 21.

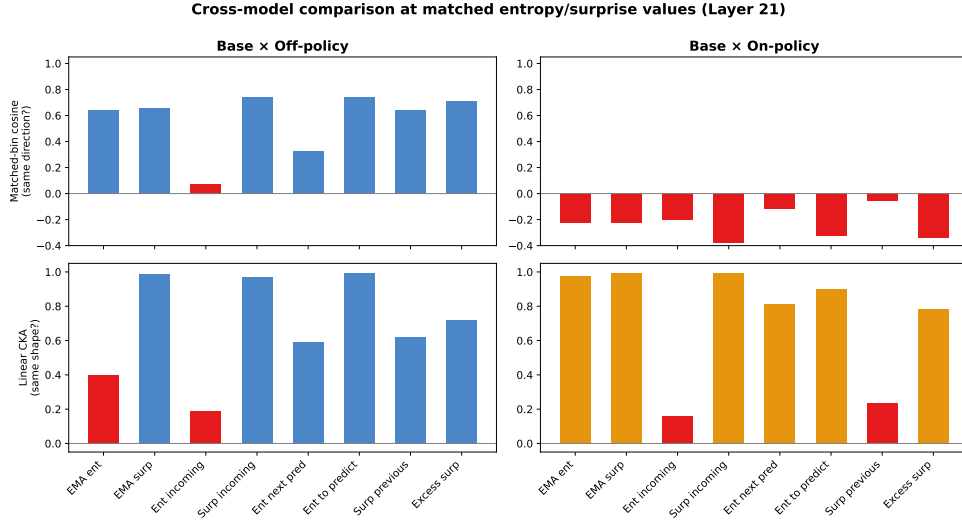


Figure 19: Cross-model comparison at layer 21. Top row: mean cosine of centered centroids at matched feature values. Bottom row: linear CKA. Left column: base vs. off-policy. Right column: base vs. on-policy. Red bars highlight outliers (cosine < 0.3 or CKA < 0.4). Off-policy, the entropy of the incoming token is the only feature with low values on both metrics. On-policy, matched-bin cosine is uniformly negative across features (directions are rotated); CKA remains high except for the entropy of the incoming token and the surprise of the previous token, whose manifold shapes also change.

exceptions to this account are incoming entropy, whose representation changes even on off-policy data, and previous-token surprise, whose manifold shape changes on on-policy data.

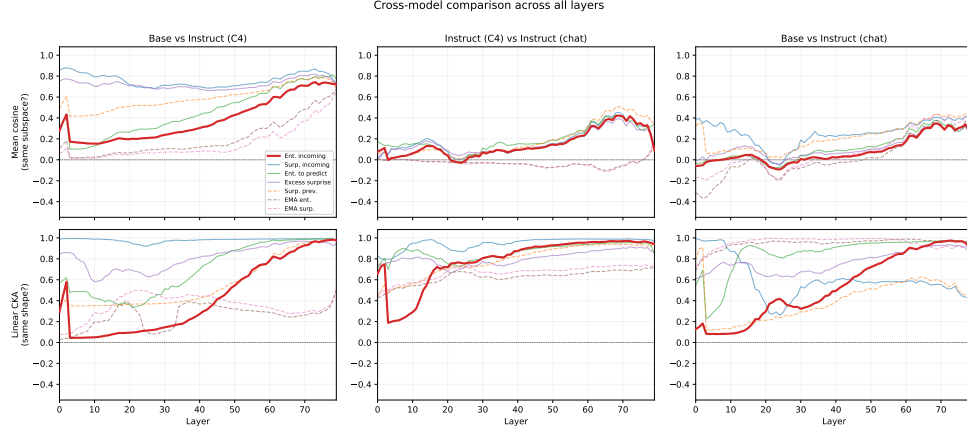


Figure 20: Cross-model comparison across all layers. Top: mean cosine. Bottom: linear CKA. Entropy of incoming token (red, bold) highlighted. Left: base vs off-policy. Center: off-policy vs on-policy. Right: base vs on-policy.

C.4 On-policy entropy representations are not causal

A quantity being represented in model activations does not imply that this representation is causal. We tested whether the on-policy entropy centroid direction (chat H_{t+1} , $k=0$) causally modulates output entropy by steering toward each of 20 bins at $\text{frac}=1.5$ across layers 4–20 on Llama-3.1-70B-Instruct, and measuring output entropy at the summary position on 8 chat contexts (assistant-field, teacher-forced) and 8 C4 contexts (teacher-forced, no template). Across the full bin range (0–0.93 nats), output entropy moved by only 0.04 nats on chat and 0.06 nats on C4, with fit slopes of 0.011 and 0.035 respectively (Figure 21). A genuinely causal direction would produce a slope close to 1. Over a comparable bin range, steering the surprise centroids moves output entropy by roughly the magnitude of the bin itself (Figure 12 in Section 2.5). We therefore conclude that the model’s output entropy is not substantially causally dependent on the internal representation of entropy that we identified, but *is* substantially modified by its internal representation of surprise.

Steering toward on-policy entropy centroids leaves output entropy unchanged

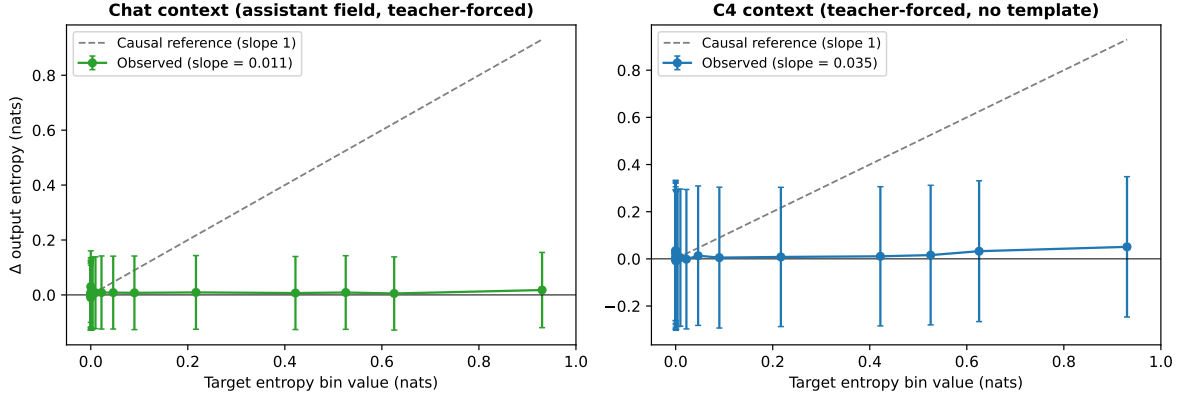


Figure 21: Steering toward the on-policy entropy centroids (chat H_{t+1} , $k=0$) at $\text{frac}=1.5$ across layers 4–20 has no substantive effect on output entropy. Each point is the mean change in output entropy at the summary position when steering toward one of 20 quantile bins (target bin value on the x-axis), averaged over 8 contexts; error bars show one standard deviation across contexts. The grey dashed line is the slope-1 reference, i.e. the response a causal direction would produce. Left: chat context (assistant field). Right: C4 context (no chat template). Both observed slopes are within 4% of zero.